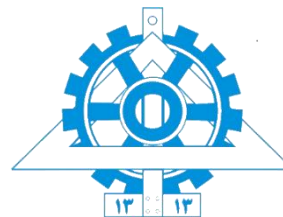




تمرین برنامه نویسی شماره ۴



عنوان: پیاده‌سازی Mini-TCP روی UDP/IP و تحلیل کنترل ازدحام

درس: شبکه‌های کامپیوتری

استاد راهنما: دکتر ناصر یزدانی^۱

مسئول دستیاران: کاظم عیارزاده^۲

دستیار آموزشی: فرشته باقری^۳

نیمسال دوم سال تحصیلی ۱۴۰۴-۰۵

^۱ نشانی پست الکترونیکی: yazdani@ut.ac.ir

^۲ نشانی پست الکترونیکی: ayarzadeh.km@ut.ac.ir

^۳ نشانی پست الکترونیکی: fereshtebaghi81@ut.ac.ir



فهرست مطالب

۳	هدف پروژه
۴	ابزار مورد نیاز
۵	مقدمه
۶	قالب کلی پروژه
۷	فاز صفر: طراحی اولیه و ساختار کد
۸	فاز یک: ارسال قابل اعتماد با Stop-and-Wait
۹	فاز دو: پیاده‌سازی Sliding Window
۱۰	فاز سه: شبیه‌سازی loss و delay در برنامه
۱۱	فاز چهار: کنترل ازدحام با Slow Start و Congestion Avoidance
۱۳	فاز پنج: تحلیل عملکرد و رسم نمودارها
۱۴	فاز امتیازی: Fast Recovery و Fast Retransmit
۱۵	نکات پیاده‌سازی
۱۶	گزارش پروژه
۱۷	معیارهای نمره‌دهی
۱۸	نحوه تحویل

هدف پروژه

در این پروژه قصد داریم یک نسخه ساده‌شده و آموزشی از پروتکل TCP را در سطح برنامه و با استفاده از زبان C++ پیاده‌سازی کنیم. در TCP واقعی، بسیاری از جزئیات مربوط به ارسال قابل‌اعتماد داده، شماره‌گذاری بایت‌ها، دریافت ACK، ارسال مجدد بسته‌های گم‌شده، کنترل جریان و کنترل ازدحام توسط سیستم‌عامل انجام می‌شود. در این تمرین، هدف این است که دانشجویان این مفاهیم را به‌صورت عملی پیاده‌سازی و تحلیل کنند.

برای ساده‌تر شدن پیاده‌سازی، در این پروژه از UDP socket استفاده می‌شود. یعنی برنامه‌ها از UDP برای ارسال و دریافت packet استفاده می‌کنند، اما قابلیت‌هایی مانند reliability, sequence number, acknowledgment, timeout, retransmission, sliding window و congestion control باید توسط خود دانشجویان در لایه application پیاده‌سازی شود. به این پروتکل پیاده‌سازی‌شده در این تمرین، Mini-TCP می‌گوییم.

هدف نهایی پروژه این است که دانشجویان درک کنند TCP چگونه بدون اطلاع مستقیم از وضعیت داخلی شبکه، با مشاهده ACKها، timeoutها و packet lossها، نرخ ارسال خود را تنظیم می‌کند و تلاش می‌کند نزدیک ظرفیت مناسب شبکه کار کند.

ابزار مورد نیاز

در این تمرین، پیاده‌سازی باید فقط با زبان C++ انجام شود. استفاده از کتابخانه‌های آماده برای پیاده‌سازی reliability یا congestion control مجاز نیست.

ابزارهای مورد نیاز:

- سیستم‌عامل Linux یا محیط مشابه Linux
- کامپایلر g++
- آشنایی با UDP socket programming
- ابزار make برای کامپایل راحت‌تر پروژه
- Python یا ابزارهای مشابه برای رسم نمودارها، فقط برای تحلیل خروجی‌ها
- Wireshark، به صورت اختیاری و امتیازی، برای مشاهده packetها

نمونه کامپایل:

```
g++ sender.cpp -o sender
```

```
g++ receiver.cpp -o receiver
```

یا با استفاده از Makefile:

```
make
```

نمونه اجرای برنامه:

```
./receiver 8080 output.txt
```

```
./sender 127.0.0.1 8080 input.txt
```

مقدمه

TCP یکی از مهم‌ترین پروتکل‌های لایه انتقال است و وظیفه دارد داده‌ها را به‌صورت قابل اعتماد بین دو برنامه ارسال کند. برخلاف UDP که ارسال packet را بدون تضمین تحویل، ترتیب و عدم تکرار انجام می‌دهد، TCP باید اطمینان حاصل کند که داده‌ها کامل، مرتب و بدون خطا به مقصد می‌رسند.

از طرف دیگر، TCP نباید بدون محدودیت داده وارد شبکه کند. اگر چندین فرستنده همزمان بیش از ظرفیت شبکه ارسال کنند، صف روترها پر می‌شود، packet‌ها از بین می‌روند، تأخیر افزایش پیدا می‌کند و حتی ممکن است throughput کلی شبکه کاهش یابد. این وضعیت congestion نام دارد.

در اسلایدهای درس بیان شد که TCP باید به‌صورت adaptive عمل کند؛ یعنی اگر شبکه ظرفیت آزاد دارد، نرخ ارسال را افزایش دهد و اگر نشانه‌ای از ازدحام مشاهده شد، نرخ ارسال را کاهش دهد. TCP این کار را معمولاً با استفاده از متغیری به نام congestion window یا cwnd انجام می‌دهد. در این تمرین، شما باید یک پروتکل ساده‌شده شبیه TCP را روی UDP پیاده‌سازی کنید و رفتار آن را در شرایط مختلف بررسی کنید.

قالب کلی پروژه

برنامه شما باید شامل دو بخش اصلی باشد:

sender.cpp

receiver.cpp

فرستنده فایل ورودی را خوانده، آن را به segmentهای کوچک تقسیم می‌کند و با استفاده از UDP برای گیرنده می‌فرستد. گیرنده segmentها را دریافت کرده، ترتیب آن‌ها را بررسی می‌کند و در نهایت فایل خروجی را بازسازی می‌کند.

برای هر segment باید یک header طراحی شود. این header حداقل باید شامل موارد زیر باشد:

```
struct PacketHeader {  
    uint32_t seq_num;  
    uint32_t ack_num;  
    uint16_t length;  
    uint16_t checksum;  
    uint8_t flags;  
};
```

فیلد flags می‌تواند برای مشخص کردن نوع packet استفاده شود. برای مثال:

DATA

ACK

FIN

در این پروژه نیازی به پیاده‌سازی کامل TCP واقعی، سه مرحله‌ای بودن handshake، یا raw socket نیست. هدف، پیاده‌سازی مفاهیم اصلی TCP به صورت آموزشی است.

فاز صفر: طراحی اولیه و ساختار کد

در این فاز باید ساختار کلی پروژه را طراحی کنید. کد شما باید خوانا، ساختارمند و قابل اجرا باشد. پیشنهاد می‌شود فایل‌های پروژه به شکل زیر باشند:

sender.cpp
receiver.cpp
packet.hpp
timer.hpp
logger.hpp
Makefile

در این فاز باید موارد زیر را انجام دهید:

- تعریف قالب packet
- پیاده‌سازی توابع serialize و deserialize برای تبدیل packet به byte stream و برعکس
- پیاده‌سازی checksum ساده برای تشخیص خطای packet
- پیاده‌سازی خواندن فایل در sender
- پیاده‌سازی نوشتن فایل در receiver
- ایجاد Makefile برای کامپایل پروژه

در گزارش این بخش توضیح دهید که header طراحی شده چه فیلدهایی دارد و هر فیلد چه کاربردی دارد.

فاز یک: ارسال قابل اعتماد با Stop-and-Wait

در این فاز باید ساده‌ترین روش ارسال قابل اعتماد را پیاده‌سازی کنید. در روش Stop-and-Wait، فرستنده در هر لحظه فقط یک packet ارسال می‌کند و تا زمانی که ACK مربوط به آن packet را دریافت نکرده است، packet بعدی را ارسال نمی‌کند.

مراحل مورد انتظار:

۱. sender فایل ورودی را به packetهای کوچک تقسیم کند.
 ۲. برای هر packet یک seq_num قرار دهد.
 ۳. receiver پس از دریافت packet، در صورت صحیح بودن checksum، یک ACK مناسب ارسال کند.
 ۴. sender پس از دریافت ACK، packet بعدی را ارسال کند.
 ۵. اگر ACK در زمان مشخص دریافت نشد، sender همان packet را دوباره ارسال کند.
- در این فاز باید timeout ساده پیاده‌سازی شود. برای مثال می‌توانید مقدار timeout را در ابتدا ثابت در نظر بگیرید: $RTO = 1000\text{ ms}$

خروجی مورد انتظار:

- فایل خروجی در سمت receiver باید دقیقاً با فایل ورودی برابر باشد.
- تعداد packetهای ارسال شده ثبت شود.
- تعداد retransmissionها ثبت شود.
- زمان کل انتقال فایل گزارش شود.

در گزارش این فاز، با یک فایل نمونه نشان دهید که ارسال فایل به درستی انجام شده است.

فاز دو: پیاده‌سازی Sliding Window

در این فاز باید روش Stop-and-Wait را به Sliding Window تبدیل کنید. در Stop-and-Wait، در هر لحظه فقط یک packet در شبکه وجود دارد و این باعث استفاده ضعیف از ظرفیت شبکه می‌شود. در Sliding Window، فرستنده می‌تواند چند packet را بدون انتظار برای ACK تک‌تک آن‌ها ارسال کند.

در این فاز باید یک متغیر به نام window_size داشته باشید. این متغیر مشخص می‌کند که حداکثر چند packet می‌توانند همزمان ارسال شده و هنوز ACK نشده باشند.

مراحل مورد انتظار:

۱. sender بتواند چند packet متوالی را ارسال کند.
 ۲. receiver برای packet‌های دریافت‌شده ACK ارسال کند.
 ۳. sender با دریافت ACK، پنجره ارسال را جلو ببرد.
 ۴. اگر timeout رخ داد، packet‌های ACK نشده دوباره ارسال شوند.
 ۵. ACK‌ها به صورت cumulative پیاده‌سازی شوند.
- در ACK تجمعی، اگر receiver تا packet شماره ۱۰ را به صورت مرتب دریافت کرده باشد، ACK شماره ۱۱ را ارسال می‌کند. یعنی اعلام می‌کند که packet بعدی مورد انتظار، packet شماره ۱۱ است.

خروجی مورد انتظار:

- مقایسه زمان انتقال فایل در حالت Stop-and-Wait و Sliding Window
- گزارش تعداد retransmission
- گزارش throughput انتقال فایل

در گزارش توضیح دهید که چرا Sliding Window معمولاً بهتر از Stop-and-Wait عمل می‌کند.

فاز سه: شبیه‌سازی loss و delay در برنامه

برای اینکه بتوانید رفتار Mini-TCP را در شرایط ازدحام بررسی کنید، باید امکان ایجاد loss و delay مصنوعی را در برنامه اضافه کنید. برای ساده نگه داشتن پروژه، می‌توانید loss را در سمت sender یا receiver شبیه‌سازی کنید. برای مثال، receiver می‌تواند با احتمال مشخصی بعضی packet‌ها را نادیده بگیرد و برای آن‌ها ACK ارسال نکند. همچنین می‌توانید برای ارسال ACK‌ها تأخیر مصنوعی قرار دهید. برنامه باید بتواند با پارامترهای زیر اجرا شود:

```
./receiver 8080 output.txt --loss 0.1 --delay 100
```

```
./sender 127.0.0.1 8080 input.txt
```

در این مثال، $\text{loss} = 0.1$ یعنی حدود ۱۰ درصد packet‌ها به‌صورت مصنوعی از بین می‌روند و $\text{delay} = 100$ یعنی receiver قبل از ارسال ACK حدود ۱۰۰ میلی‌ثانیه صبر می‌کند.

مقادیر پیشنهادی برای آزمایش:

loss = 0%, 5%, 10%, 20%

delay = 0 ms, 50 ms, 100 ms, 200 ms

خروجی مورد انتظار:

- throughput برای هر مقدار loss
- تعداد retransmission برای هر مقدار loss
- زمان کل انتقال فایل
- تحلیل اثر loss و delay روی عملکرد پروتکل

در گزارش توضیح دهید که چرا افزایش loss باعث افزایش retransmission و کاهش throughput می‌شود.

فاز چهار: کنترل ازدحام با Slow Start و Congestion Avoidance

در این فاز باید مفهوم congestion window یا cwnd را پیاده‌سازی کنید. از این مرحله به بعد، تعداد packet‌هایی که sender اجازه دارد همزمان در شبکه داشته باشد، فقط با window_size ثابت کنترل نمی‌شود، بلکه با cwnd نیز محدود می‌شود.

در هر لحظه:

$$\text{effective_window} = \min(\text{receiver_window}, \text{cwnd})$$

برای ساده‌سازی، می‌توانید receiver_window را ثابت در نظر بگیرید و تمرکز اصلی را روی cwnd قرار دهید.

ابتدا مقدار cwnd را برابر یک packet قرار دهید:

$$\text{cwnd} = 1$$

$$\text{ssthresh} = 16$$

سپس دو حالت اصلی را پیاده‌سازی کنید:

Slow Start

تا زمانی که:

$$\text{cwnd} < \text{ssthresh}$$

با دریافت هر ACK جدید، مقدار cwnd افزایش می‌یابد. در این حالت رشد cwnd تقریباً نمایی است.

نمونه رفتار مورد انتظار:

$$\text{cwnd} = 1$$

$$\text{cwnd} = 2$$

$$\text{cwnd} = 4$$

$$\text{cwnd} = 8$$

...

Congestion Avoidance

وقتی:

$$\text{cwnd} \geq \text{ssthresh}$$

پروتکل وارد مرحله Congestion Avoidance می‌شود. در این مرحله رشد cwnd باید آرام‌تر و خطی باشد.

برای ساده‌سازی می‌توانید در هر RTT تقریباً مقدار cwnd را به اندازه ۱ packet افزایش دهید.

واکنش به Timeout

اگر timeout رخ دهد، باید فرض کنید که شبکه دچار congestion شده است. بنابراین:

$$ssthresh = cwnd / 2$$

$$cwnd = 1$$

سپس پروتکل دوباره از Slow Start شروع می‌کند.

خروجی مورد انتظار:

- فایل cwnd.csv شامل تغییرات cwnd در طول زمان
- فایل events.log شامل رویدادهایی مثل ارسال packet، دریافت ACK، timeout و retransmission
- نمودار cwnd بر حسب زمان
- نمودار throughput بر حسب زمان

در گزارش این فاز توضیح دهید که چرا TCP هنگام timeout مقدار cwnd را کاهش می‌دهد و چرا دوباره از Slow Start شروع می‌کند.

فاز پنج: تحلیل عملکرد و رسم نمودارها

در این فاز باید خروجی‌های برنامه را تحلیل کنید. حداقل سه آزمایش مختلف باید انجام شود:

آزمایش اول: شبکه بدون loss

loss = 0%

delay = 0 ms

در این حالت باید بررسی کنید که cwnd چگونه رشد می‌کند و throughput نهایی چقدر است.

آزمایش دوم: شبکه با loss متوسط

loss = 5% یا 10%

delay = 50 ms

در این حالت باید بررسی کنید که چند بار timeout رخ داده و cwnd چگونه کاهش پیدا کرده است.

آزمایش سوم: شبکه با loss زیاد

loss = 20%

delay = 100 ms

در این حالت باید بررسی کنید که آیا پروتکل همچنان می‌تواند فایل را کامل منتقل کند یا خیر و throughput چقدر کاهش پیدا می‌کند.

متریک‌های مورد نیاز:

- Throughput
- تعداد packet های ارسال شده
- تعداد packet های retransmit شده
- تعداد timeout ها
- زمان کل انتقال
- نمودار cwnd/time
- نمودار throughput/time

در گزارش باید نمودارها و جداول خروجی را قرار دهید و تحلیل کنید که با افزایش loss و delay چه تغییری در عملکرد رخ می‌دهد.

فاز امتیازی: Fast Recovery و Fast Retransmit

در TCP، اگر sender چند ACK تکراری برای یک packet دریافت کند، می‌تواند قبل از timeout حدس بزند که یک packet از بین رفته است. در این حالت به جای صبر کردن تا timeout، packet گم‌شده را سریع‌تر retransmit می‌کند. به این مکانیزم Fast Retransmit گفته می‌شود. در این پروژه، اگر sender سه ACK تکراری برای یک ack_num دریافت کرد، باید packet مربوطه را دوباره ارسال کند. مکانیزم پیشنهادی:

```
if duplicate_ack_count == 3:  
    ssthresh = cwnd / 2  
    retransmit missing packet
```

در حالت ساده، می‌توانید بعد از Fast Retransmit مقدار cwnd را برابر ۱ قرار دهید. در حالت کامل‌تر، می‌توانید Fast Recovery را نیز پیاده‌سازی کنید و به جای برگشت کامل به Slow Start، مقدار cwnd را به مقدار نزدیک‌تری به ssthresh تنظیم کنید.

خروجی مورد انتظار:

- مشخص کردن مواردی که Fast Retransmit رخ داده است
- مقایسه زمان انتقال با و بدون Fast Retransmit
- مقایسه تعداد timeoutها با و بدون Fast Retransmit
- رسم نمودار cwnd و مشخص کردن نقاط Fast Retransmit روی نمودار

نکات پیاده‌سازی

در این پروژه، رعایت نکات زیر الزامی است:

- استفاده از TCP socket برای انتقال داده اصلی مجاز نیست.
- انتقال داده باید با UDP socket انجام شود.
- reliability, ACK, retransmission, sliding window و congestion control باید توسط خود دانشجو پیاده‌سازی شود.
- استفاده از کتابخانه آماده برای پیاده‌سازی TCP یا reliable UDP مجاز نیست.
- کد باید ساختارمند، خوانا و قابل اجرا باشد.
- فایل خروجی receiver باید دقیقاً با فایل ورودی sender برابر باشد.
- برنامه باید برای فایل‌های متنی و باینری قابل استفاده باشد.
- پارامترهایی مانند delay, loss, packet size و timeout باید قابل تنظیم باشند.

گزارش پروژه

علاوه بر کد، هر گروه باید یک گزارش PDF تحویل دهد. گزارش باید شامل موارد زیر باشد:

- توضیح معماری کلی پروژه
- توضیح قالب packet و فیلدهای header
- توضیح نحوه کار Stop-and-Wait
- توضیح نحوه کار Sliding Window
- توضیح نحوه پیاده‌سازی timeout و retransmission
- توضیح نحوه پیاده‌سازی Slow Start و Congestion Avoidance
- توضیح مفهوم cwnd و ssthresh
- نحوه اجرای برنامه همراه با screenshot
- جدول نتایج آزمایش‌ها
- نمودارهای cwnd/time و throughput/time
- تحلیل اثر loss و delay بر عملکرد پروتکل
- توضیح چالش‌ها و مشکلات پیاده‌سازی
- شرح تقسیم کار بین اعضای گروه



معیارهای نمره‌دهی

بخش	نمره
طراحی packet، ساختار کد و Makefile	۱۰
پیاده‌سازی Stop-and-Wait	۱۵
پیاده‌سازی Sliding Window و ACK تجمعی	۲۰
پیاده‌سازی timeout و retransmission	۱۵
پیاده‌سازی Slow Start و Congestion Avoidance	۲۰
جمع‌آوری log، محاسبه متریک‌ها و رسم نمودارها	۱۰
گزارش کامل، تحلیل نتایج و screenshotها	۱۰
مجموع	۱۰۰

بخش امتیازی:

بخش امتیازی	نمره اضافه
پیاده‌سازی Fast Retransmit	۱۰
پیاده‌سازی Fast Recovery	۵
تحلیل با Wireshark	۵
پشتیبانی از چند connection همزمان	۱۰

نحوه تحویل

موارد مورد نیاز برای تحویل:

sender.cpp
receiver.cpp
packet.hpp
timer.hpp
logger.hpp
Makefile
report.pdf
logs/
plots/
sample_input.txt
sample_output.txt

فایل‌ها باید در قالب یک فایل فشرده ارسال شوند:

CN_CA_4_<first member last name>_<second member last name>.rar

نکات مهم:

- پروژه به صورت گروهی و حداکثر دو نفره انجام می‌شود.
- هر دو نفر باید در گزارش، شرح تقسیم کار خود را بنویسند.
- کدها باید کاملاً خوانا و ساختارمند باشند.
- هرگونه کپی برداری باعث حذف نمره خواهد شد.
- گزارش باید مانند یک document کامل باشد و فقط شامل screenshot نباشد.
- نمودارها و تحلیل‌ها بخش مهمی از نمره پروژه هستند.
- فایل خروجی receiver باید با فایل ورودی sender مقایسه شود و نتیجه صحت انتقال در گزارش آورده شود.
- هدف این تمرین یادگیری عمیق مفاهیم TCP و congestion control است؛ لطفاً پروژه را خودتان انجام دهید.

موفق باشید.